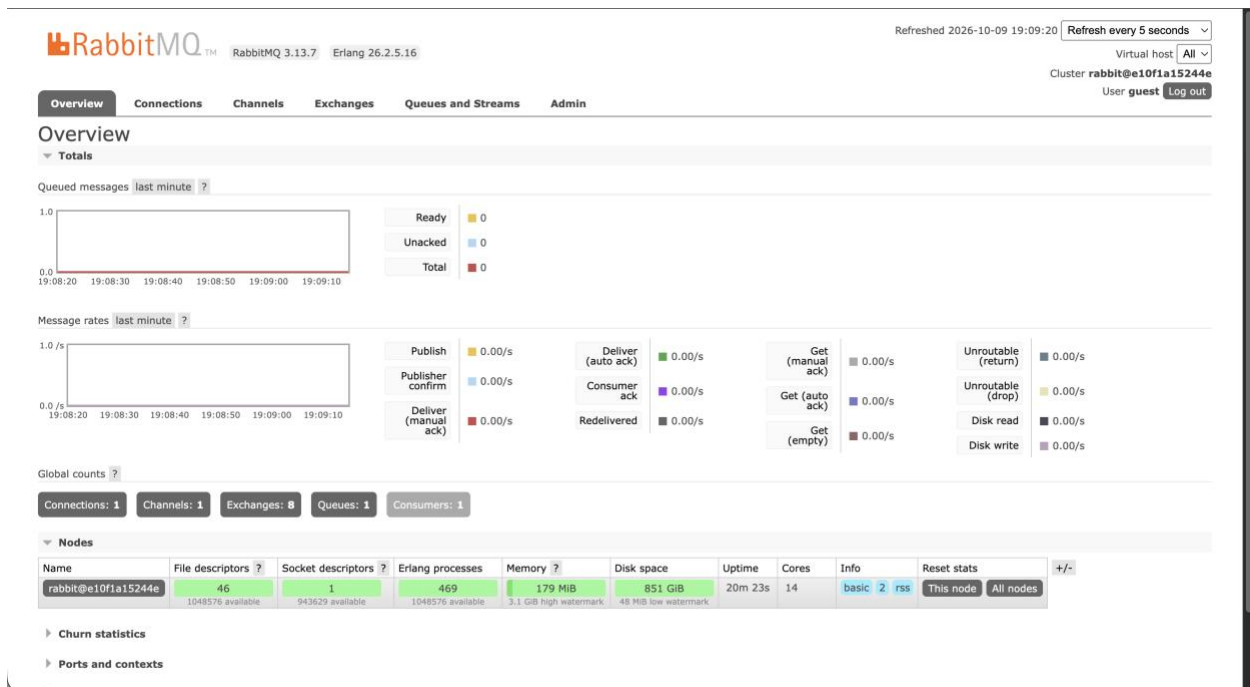




Cameron Elia - Module 6 Grad School Cafe Data Analysis

Analysis

[Pull Data](#) [Update Analysis](#)

Pull Data enqueues a background task that checks Grad Cafe for newly submitted application results and adds new records to the database. Update Analysis recomputes all statistics from the current database contents. Both operations run asynchronously via the worker service.

Analytics recompute queued. Data will be refreshed shortly.

Q1: How many entries in your database are from applicants who applied for Fall 2026?
Answer: Applicant count: 32544

Q2: Among entries that provide a nationality classification, what percentage are international students?
Answer: International percentage: 46.31%

Q3: What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?
Answer: Average GPA: 3.76, Average GRE: 165.67, Average GRE V: 160.41, Average GRE AW: 4.32

Q4: What is the average GPA of American applicants who applied for Fall 2026?
Answer: Average GPA American: 3.79

Q5: What percentage of Fall 2025 entries are acceptances?
Answer: Percentage accepted: 39.87%

Q6: What is the average GPA of accepted applicants who applied for Fall 2026?
Answer: Average GPA accepted: 3.76

Docker Compose Architecture

The `module\_6` stack is defined in `docker-compose.yml` and consists of five services running on a shared Docker network:

Service	Image	Role
db	postgres:16	Stores applicant records and analytics data

Service	Image	Role
rabbitmq	rabbitmq:3.13-management	Message broker between the web and worker services
loader	module_6-web (one-shot)	Seeds the database with applicant_data.json on startup, then exits
web	module_6-web	Serves the Flask analysis page on port 8080
worker	module_6-worker	Consumes tasks from RabbitMQ and performs all database writes

`web` and `worker` services both depend on `db` and `rabbitmq` reaching a healthy state before starting, enforced with `condition: service\_healthy`. PostgreSQL data persists across restarts via the named volume `pgdata`.

RabbitMQ Message Flow

Clicking either button on the analysis page triggers the following sequence:

1. The Flask route calls `publish\_task(kind)` in `publisher.py`.
2. `publisher.py` opens a connection to RabbitMQ using `RABBITMQ\_URL`, declares a durable direct exchange `tasks`, a durable queue `tasks\_q`, and a binding with routing key `tasks`. Each step is idempotent.
3. A compact JSON message `{"kind": "...", "ts": "<UTC ISO>", "payload": {}}` is published with `delivery\_mode=2` (persistent) so it survives a broker restart.
4. The connection is closed immediately and the Flask route returns HTTP 202. The request completes without waiting for the worker.
5. The worker's `consumer.py` receives the message via `basic\_consume`. With `basic\_qos(prefetch\_count=1)`, the worker processes one message at a time.
6. The message `kind` is routed through `TASK\_MAP` to the appropriate handler.
7. On success, the database transaction is committed and the message is acked. On error, the transaction is rolled back and the message is nacked with `requeue=False` to prevent infinite retry loops.

Database Initialization

When ``docker compose up`` runs:

1. The ``db`` service mounts ``src/db/init.sql`` into PostgreSQL's ``docker-entrypoint-initdb.d/`` directory, which creates the ``applicants`` and ``ingestion_watermarks`` tables automatically on first start.
2. Once ``db`` is healthy, the ``loader`` service starts and runs ``load_data.py`` against ``src/data/applicant_data.json``, inserting all applicant records via upsert. The loader exits after completing.

Subsequent ``docker compose up`` calls skip schema creation and re-run the loader as an upsert, updating existing rows without duplicating them.

Worker Behavior

The worker service runs ``consumer.py`` as a long-lived process. It implements two task handlers:

- ``handle_scrape_new_data``: reads ``applicant_data.json`` and loads only records whose ``p_id`` exceeds the value stored in the ``ingestion_watermarks`` table, which is the watermark. The caller (the message loop) commits the transaction only after all inserts and the watermark update succeed.
 - Records are inserted with ``ON CONFLICT (p_id) DO NOTHING`` for idempotence
 - After a successful batch insert, the watermark is advanced to the highest ``p_id`` seen
- ``handle_recompute_analytics``: re-executes all analytics queries within the same per-message transaction, exercising the current database contents. The web service reads results directly from the database on the next page load.

Verification Steps

1. ``docker compose up --build``: all five services start; loader exits cleanly after seeding.
2. Visit <http://localhost:8080/analysis>: the analysis page loads with applicant statistics drawn live from PostgreSQL.
3. Visit <http://localhost:15672>: RabbitMQ management UI is reachable (credentials: guest / guest).

4. Click "Pull Data": the page shows "Pull queued." The worker logs "Scraped N new record(s)".
5. Click "Update Analysis": the page shows "Analytics recompute queued." The worker logs "Analytics recomputed".

Docker Hub Registry

Web image: https://hub.docker.com/r/celajh/module_6_web

Worker image: https://hub.docker.com/r/celajh/module_6_worker